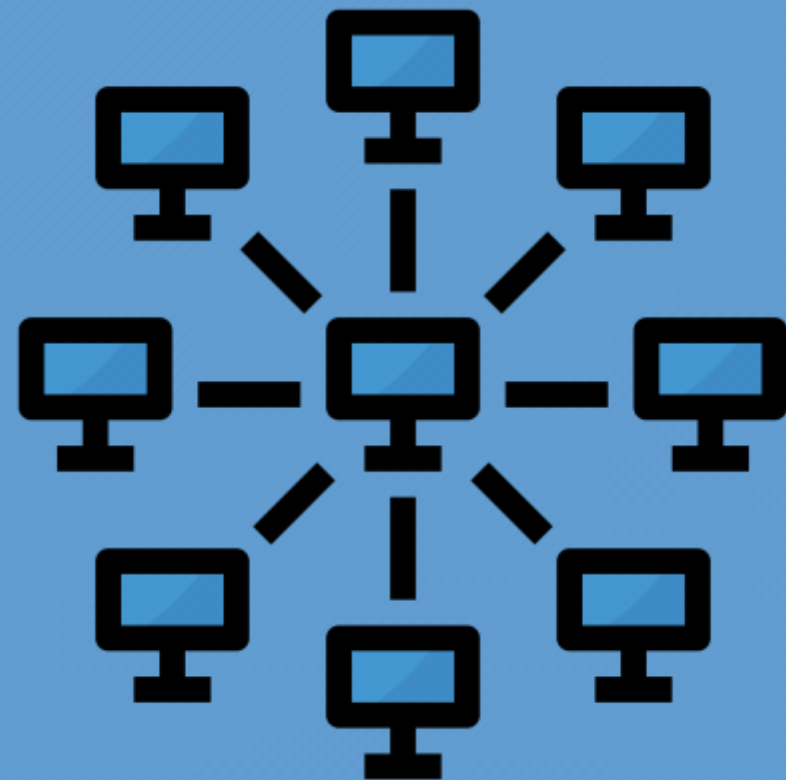


ОСНОВЫ СЕТЕВЫХ ТЕХНОЛОГИЙ. ЧАСТЬ 1



ЛЕКЦИЯ 15. ПРОТОКОЛ HTTP

КАФЕДРА
ТЕЛЕКОММУНИКАЦИЙ

15.1. ПРОТОКОЛ HTTP

15.1.1. ОСНОВЫ ПРОТОКОЛА HTTP

UNIX и Linux являются доминирующими платформами для обслуживания веб-приложений. По данным сайта w3techs.com, 67% ведущих веб-сайтов, занимающих первый миллион мест, обслуживаются операционной системой Linux или FreeBSD. Кроме того, программное обеспечение веб-серверов с открытым исходным кодом занимает более 80% рынка.

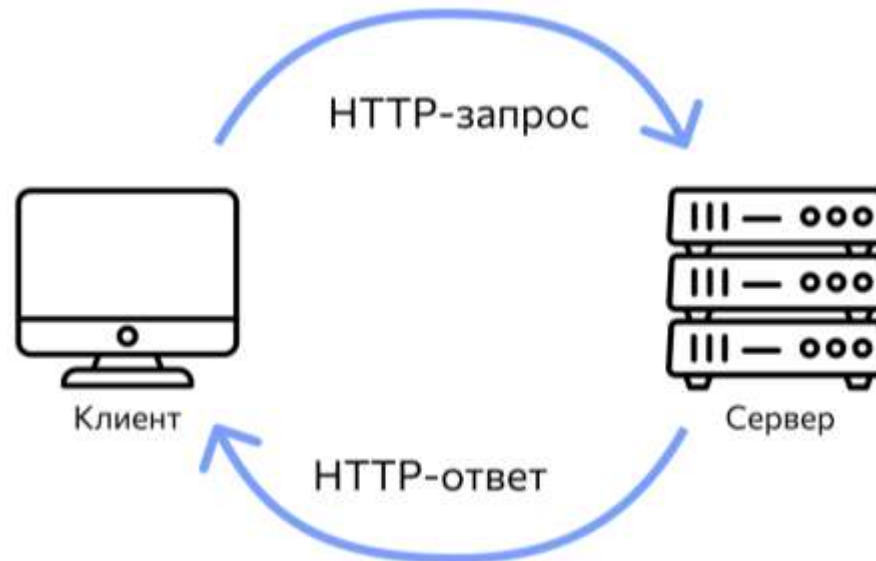


15.1. ПРОТОКОЛ HTTP

15.1.1. ОСНОВЫ ПРОТОКОЛА HTTP

Прикладной протокол **HyperText Transfer Protocol** – протокол передачи гипертекста. Глубокое понимание протокола HTTP является основной обязанностью всех системных администраторов.

В своей простейшей форме HTTP – это клиент-серверный протокол с одним запросом и одним ответом. Клиенты, также называемые пользовательскими агентами, отправляют HTTP-серверу запросы на ресурсы.



15.1. ПРОТОКОЛ HTTP

15.1.1. ОСНОВЫ ПРОТОКОЛА HTTP

Серверы получают входящие запросы и обрабатывают их, получая файлы с локальных дисков, повторно отправляя запросы на другие серверы, запрашивая базы данных или выполняя любое количество других возможных вычислений.

Типичный просмотр одной веб-страницы в Интернете предполагает десятки или сотни таких обменов. В версиях протокола **HTTP 1.0** и **1.1** стороны обмениваются информацией в виде обычного текста.

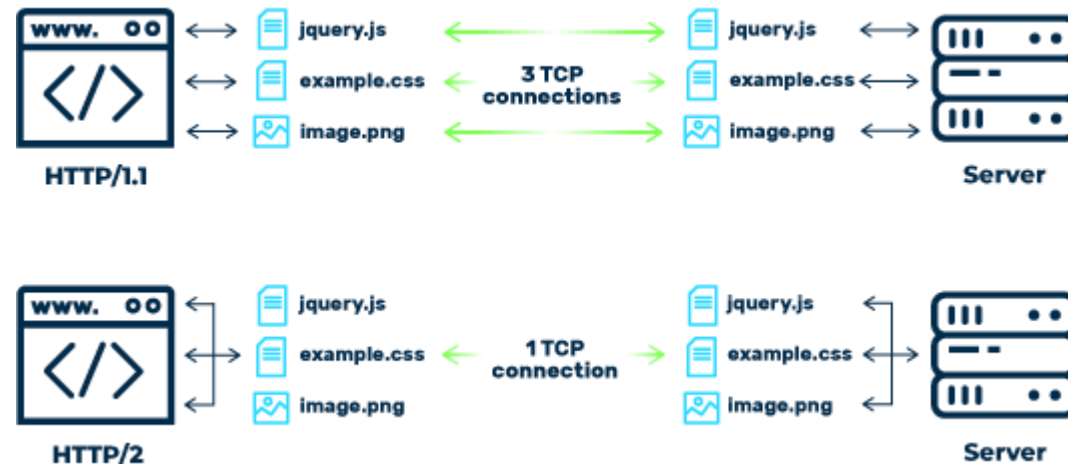
Администраторы могут взаимодействовать с серверами напрямую, запустив утилиты `telnet` или `netcat`. Они также могут наблюдать и собирать обмены данными по протоколу HTTP с помощью такого программного обеспечения для сбора пакетов, как `tcpdump`, независимого от используемого протокола.

15.1. ПРОТОКОЛ HTTP

15.1.1. ОСНОВЫ ПРОТОКОЛА HTTP

Сейчас веб-сообщество находится в процессе внедрения **HTTP/2**, основной версии протокола, которая сохраняет совместимость с предыдущими версиями, но предоставляет ряд новых решений для повышения производительности.

Стремясь обеспечить универсальное использование протокола HTTPS (защищенного, зашифрованного протокола HTTP) для следующего поколения Интернета, основные браузеры, такие как Firefox и Chrome, решили поддерживать HTTP/2 только через **TLS-зашифрованные соединения**.



15.1. ПРОТОКОЛ HTTP

15.1.1. ОСНОВЫ ПРОТОКОЛА HTTP

Чтобы упростить синтаксический анализ и повысить эффективность сети, протокол HTTP/2 переводится из текстового формата в двоичный. Семантика HTTP остается прежней, но поскольку переданные данные больше не поддаются непосредственному анализу людьми, универсальные средства, такие как `telnet`, теперь бесполезны.

Восстановить определенную интерактивность и возможность отладки соединений HTTP/2 помогает удобная утилита командной строки `h2i`, входящая в состав сетевого репозитория языка Go. Многие HTTP-специфичные инструменты, такие как `curl`, также изначально поддерживают протокол HTTP/2.



15.1. ПРОТОКОЛ HTTP

15.1.1. ОСНОВЫ ПРОТОКОЛА HTTP

Используемый формат для протокола HTTP – ASCII-текст, разделяемый символами <CR><LF>, может содержать двоичные данные (в теле запроса/ответа). Не имеет встроенных средств шифрования.

Использует протокол **TCP** и порт 80/TCP (также могут использоваться и другие порты, прежде всего 8000 и 8080).

Изначально был создан для передачи гипертекста (файлов **HTML** – HyperText Markup Language), но успешно используется для передачи текстовых (json, xml) и двоичных (аудио, видео и т. д.) данных.

```
<!DOCTYPE html>
<html>
<!-- created 2010-01-01 -->
<head>
  <title>sample</title>
</head>
<body>
  <p>Voluptatem accusantium
    totam rem aperiam.</p>
</body>
</html>
```

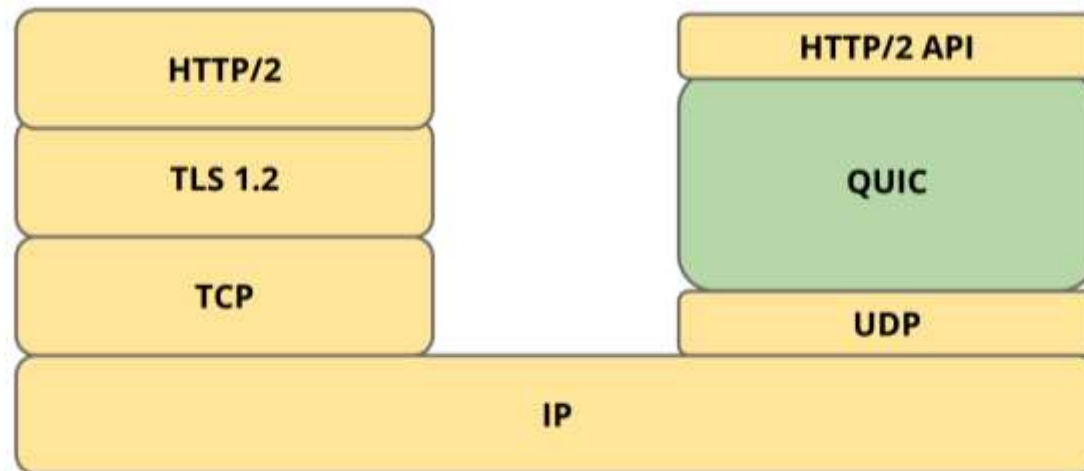
HTML

15.1. ПРОТОКОЛ HTTP

15.1.1. ОСНОВЫ ПРОТОКОЛА HTTP

Рассматриваемая нами версия протокола – **HTTP/1.1**. Также протокол дополняется другим протоколом – WebSocket (и WebSocketSecure).

QUIC – экспериментальный интернет-протокол, разработанный Google в конце 2012 года. QUIC позволяет мультиплексировать несколько потоков данных между двумя компьютерами, работая поверх протокола UDP, и содержит возможности шифрования, эквивалентные TLS и SSL. Имеет более низкую задержку соединения и передачи, чем TCP.



15.1. ПРОТОКОЛ HTTP

15.1.2. ФОРМАТ ПРОТОКОЛА HTTP

Протокол позволяет работать в режиме запрос-ответ. HTTP-запросы и ответы похожи по структуре. После начальной строки оба содержат последовательность заголовков, пустую строку и, наконец, тело сообщения, называемое полезной нагрузкой.

Первая строка запроса указывает действие, которое должен выполнить сервер. Она состоит из метода запроса (также известного как глагол), пути для выполнения действия и используемой версии HTTP.

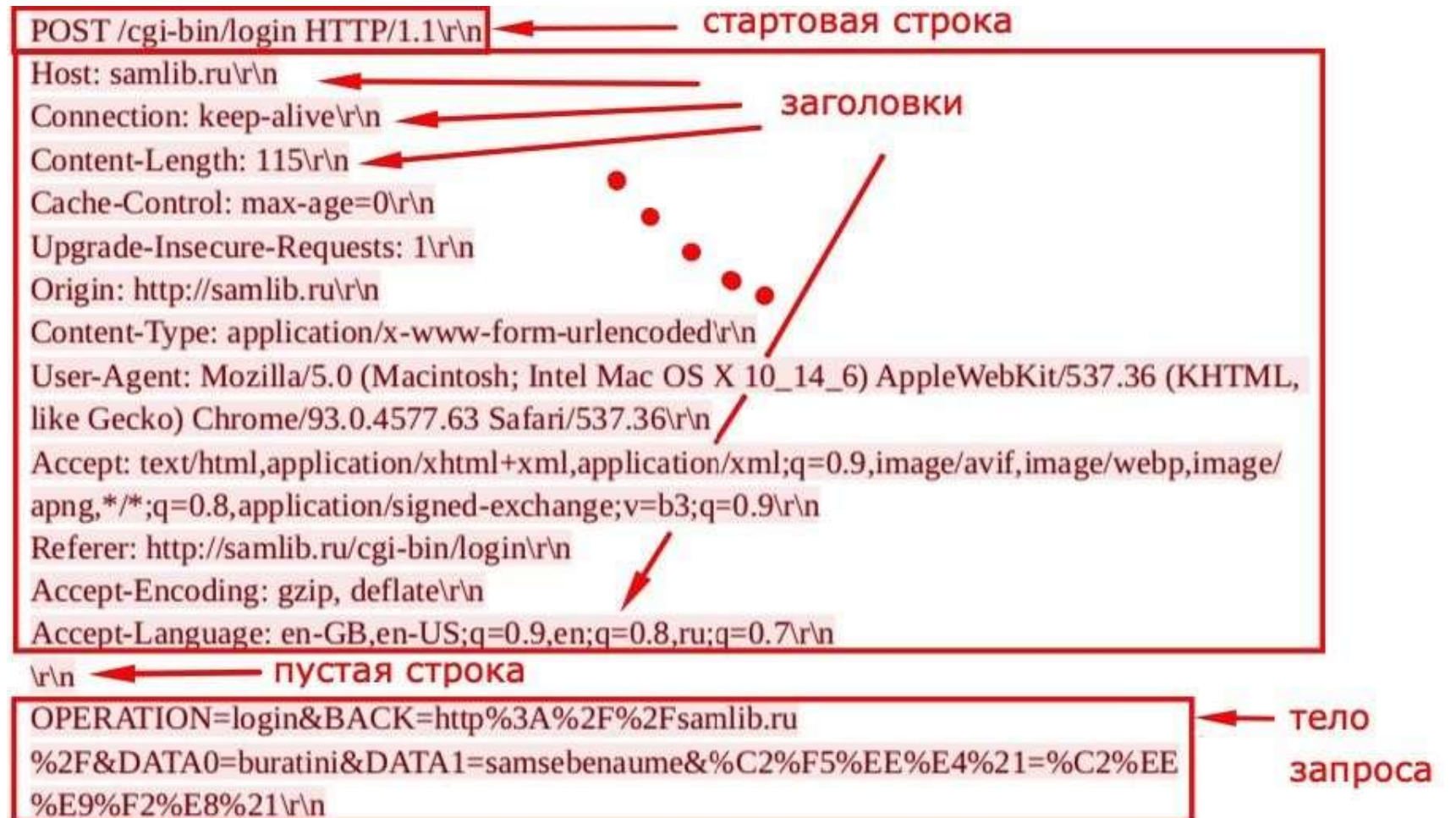
Структура формата сообщения HTTP (как запроса, так и ответа):

1. Стартовая строка <CR><LF>.
2. Заголовки (1 и более) в формате «ключ: значение», разделенные <CR><LF> (\r\n).
3. Тело сообщения (запроса или ответа), отделяется от последнего заголовка пустой строкой (<CR><LF><CR><LF>).
4. Тело сообщения может быть пустым.

15.1. ПРОТОКОЛ HTTP

15.1.2. ФОРМАТ ПРОТОКОЛА HTTP

Пример тела запроса:



15.1. ПРОТОКОЛ HTTP

15.1.2. ФОРМАТ ПРОТОКОЛА HTTP

Простой пример запроса с пустым телом:

GET / HTTP/1.1\r\n ← Стартовая строка
Host: samlib.ru\r\n ← Заголовки (один заголовок)
\r\n ← Пустая строка
← Тело запроса пустое (отсутствует)

Запрос на получение HTML-страницы верхнего уровня может выглядеть так: GET /index.html HTTP/1.1

Основные методы
HTTP-запроса:



Глагол	Безопасный?	Цель
GET	Да	Получает указанный ресурс
HEAD	Да	Аналогичен GET, но не требует никакой нагрузки; извлекает только метаданные
DELETE	Нет	Удаляет указанный ресурс
POST	Нет	Применяет данные запроса к данному ресурсу
PUT	Нет	Аналогичен POST, но подразумевает замену существующего содержимого
OPTIONS	Да	Показывает, какие методы поддерживает сервер для указанного пути

15.1. ПРОТОКОЛ HTTP

15.1.2. ФОРМАТ ПРОТОКОЛА HTTP

Начальная строка в ответе, называемая строкой состояния, указывает текущую ситуацию с запросом. Это выглядит так: HTTP/1.1 200 OK. Важной частью является трехзначный цифровой **код состояния**. Последующая фраза – это полезный комментарий на английском языке, который программное обеспечение игнорирует.

Первая цифра в коде определяет его класс, т.е. общий характер результата. В таблице показаны пять определенных классов. Остальные две цифры в классе дают дополнительную информацию. В протоколе определено более 60 кодов состояния, но только некоторые из них обычно встречаются на практике.

Код	Общие указания	Примеры
1xx	Полученный запрос; обработка продолжается	101 Switching protocols
2xx	Успех	200 OK 201 Created
3xx	Необходимы дополнительные действия	301 Moved permanently 302 Found ^a
4xx	Недопустимый запрос	403 Forbidden 404 Not Found
5xx	Ошибка сервера или среды выполнения запроса	503 Service Unavailable

^aЧаще всего используется (хотя и нецелесообразно, если следовать спецификации) для временных переадресаций.

15.1. ПРОТОКОЛ HTTP

15.1.2. ФОРМАТ ПРОТОКОЛА HTTP

Пример ответа:

The diagram illustrates the structure of an HTTP response. It shows a sequence of lines representing the response body. Red arrows and labels point to specific parts of the response:

- Стартовая строка** (Start line): Points to the first line, `HTTP/1.1 200 OK\r\n`.
- Заголовки** (Headers): A bracket groups the following five lines: `Server: nginx/1.7.9\r\n`, `Date: Mon, 22 Nov 2021 07:08:43 GMT\r\n`, `Content-Type: text/html; charset=windows-1251\r\n`, `Transfer-Encoding: chunked\r\n`, and `Connection: keep-alive\r\n`.
- пустая строка** (Empty line): Points to the line `Last-Modified: Mon, 22 Nov 2021 07:08:19 GMT\r\n`.
- пустая строка** (Empty line): Points to the line `\r\n`.
- Тело ответа** (Response body): A bracket groups the following HTML content: `<html>\r\n`, `<head>\r\n`, `<title>Журнал "Самиздат";</title>\r\n`, `<link rel="openid.server" href="http://samlib.ru/cgi-bin/oid_login" />\r\n`, `<link rel="openid2.provider" href="http://samlib.ru/cgi-bin/oid_login" />\r\n`, `</head>\r\n`, `<body bgcolor="#E9E9E9">\r\n`, `<div align="right">\r\n`, `<h3></h3>\r\n`, and `</div>\r\n`.

```
HTTP/1.1 200 OK\r\nServer: nginx/1.7.9\r\nDate: Mon, 22 Nov 2021 07:08:43 GMT\r\nContent-Type: text/html; charset=windows-1251\r\nTransfer-Encoding: chunked\r\nConnection: keep-alive\r\nLast-Modified: Mon, 22 Nov 2021 07:08:19 GMT\r\n\r\n<html>\r\n<head>\r\n<title>Журнал &quot;Самиздат&quot;;</title>\r\n<link rel="openid.server" href="http://samlib.ru/cgi-bin/oid_login" />\r\n<link rel="openid2.provider" href="http://samlib.ru/cgi-bin/oid_login" />\r\n</head>\r\n<body bgcolor="#E9E9E9">\r\n<div align="right">\r\n<h3></h3>\r\n</div>\r\n.....
```

15.1. ПРОТОКОЛ HTTP

15.1.3. СТРУКТУРА ОТВЕТА (ПРИМЕРЫ)

Пример кода группы 1XX – 101 – Switching Protocols – при переходе на WebSocket.

Примеры кодов группы 2XX – 200 Ok – успешный результат, 201 – Created – успешное создание ресурса (в заголовке Location сообщается адрес созданного ресурса).

Пример кода группы 3XX – 301 Moved Permanently (в заголовке Location сообщается адрес перенаправления). Служит для переадресации с http на https, с десктопной версии на мобильную (если запрос пришел с мобильного устройства) и т.д.

Примеры кодов группы 4XX – 404 Not Found – ресурс не найден, 403 Forbidden – доступ запрещен, 401 – Unauthorized – пользователь не авторизован.

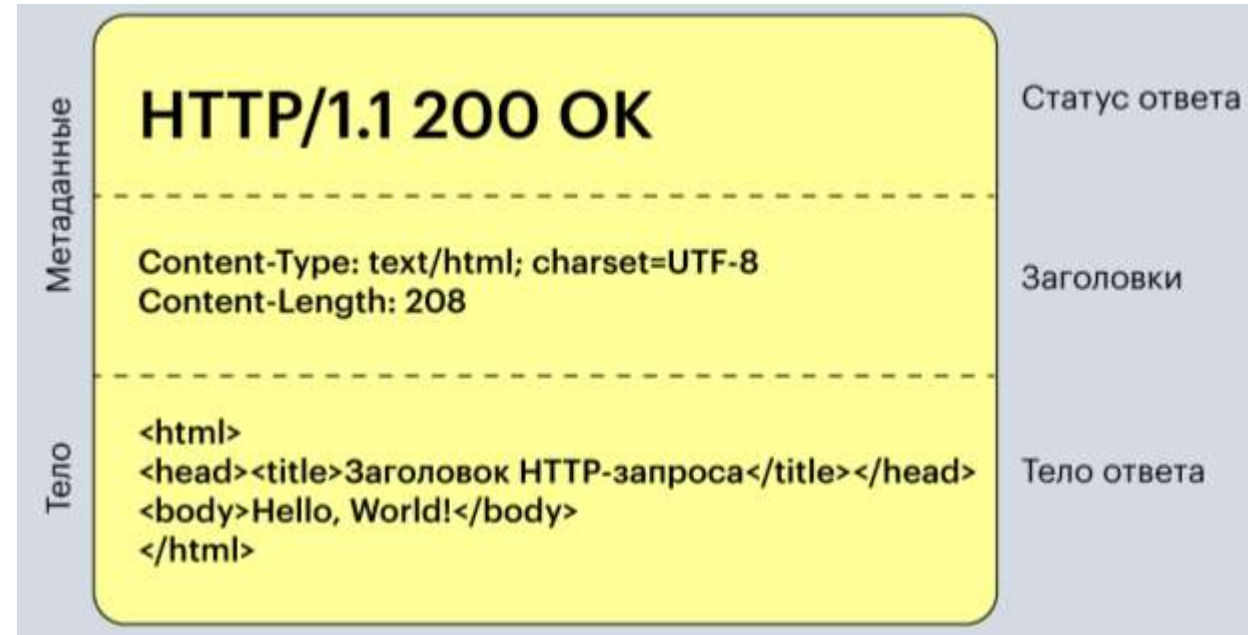
Примеры кодов группы 5xx – 500 Internal Server Error – внутренняя ошибка сервера (например, ошибка в файле .htaccess для сервера Apache2), 502 Bad Gateway – балансировщик/реверс-прокси попытался обратиться к серверу, выполняющему веб-сценарии, но получил недействительный ответ.

15.1. ПРОТОКОЛ HTTP

15.1.4. ЗАГОЛОВКИ И ТЕЛО СООБЩЕНИЯ

Заголовки указывают **метаданные** о запросе или ответе (например, о разрешении сжатия), какие типы содержимого принимаются, ожидаются или предоставляются и как промежуточные кэши должны обрабатывать данные.

Для запросов единственным требуемым заголовком является `Host`, который используется программным обеспечением веб-сервера для определения того, к какому именно сайту происходит обращение. Основные заголовки приведены в таблице далее.



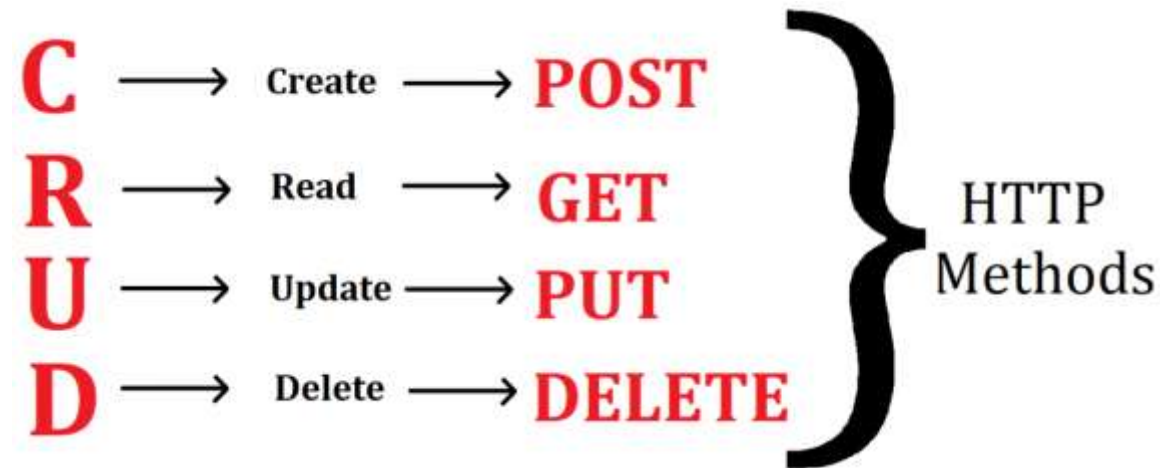
15.1. ПРОТОКОЛ HTTP

15.1.4. ЗАГОЛОВКИ И ТЕЛО СООБЩЕНИЯ

Фактически обе стороны транзакции могут включать любые заголовки, которые они пожелают. Обе стороны должны игнорировать заголовки, которые они не понимают. Заголовки отделяются от тела сообщения пустой строкой.

Для запросов тело сообщения может включать параметры (для **POST** или **PUT**) или содержимое загружаемого файла. Для ответов тело сообщения является полезной нагрузкой запрашиваемого ресурса (например, файл в формате HTML, данные изображения или результаты запроса).

Тело сообщения не обязательно читается человеком, поскольку оно может содержать изображения или другие двоичные данные. Тело также может быть пустым.



15.1. ПРОТОКОЛ HTTP

15.1.4. ЗАГОЛОВКИ И ТЕЛО СООБЩЕНИЯ

Имя: пример	Направление ^a	Содержимое
Host: www.admin.com	→	Имя домена и запрашиваемый порт
Content-Type: application/json	↔	Желаемый или существующий формат данных
Authorization: Basic QWx...FtZ==	→	Сертификаты для базовой аутентификации протокола HTTP
Last-Modified: Wed, Sep 7 2016...	←	Последняя известная дата модификации объекта
Cookie: flavor=oatmeal	→	Объект cookie, возвращаемый пользовательским агентом
Content-Length: 423	↔	Длина тела в байтах
Set-Cookie: flavor=oatmeal	←	Объект cookie, сохраняемый пользовательским агентом
User-Agent: curl/7.37.1	→	Пользовательский агент, пославший запрос
Server: nginx/1.6.2	←	Программное обеспечение сервера, отвечающее на запрос
Upgrade: HTTP/2.0	↔	Запрос на переход к другому протоколу
Expires: Sat, 15 Oct 2016 14:02:...	←	Допустимая продолжительность кеширования ответа
Cache-Control: max-age=7200	↔	Аналогичен Expires, но допускает больше контроля

^a Направление: → только запрос, ← только ответ или ↔ и запрос, и ответ.

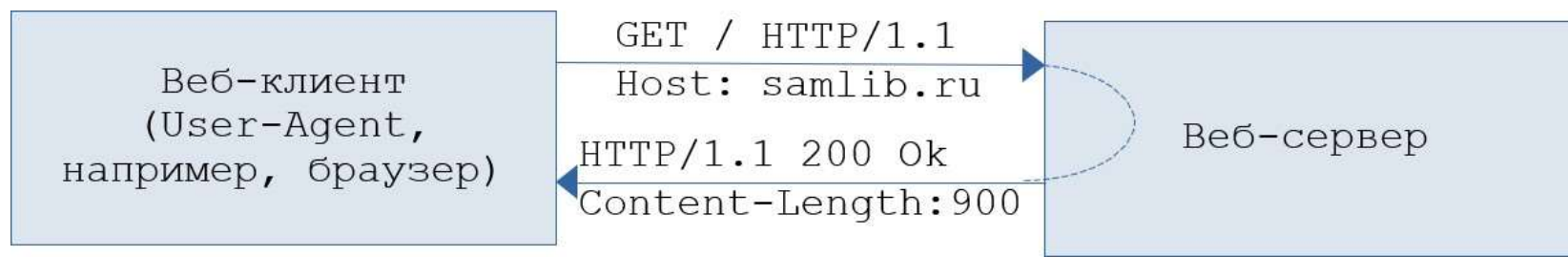
15.1. ПРОТОКОЛ HTTP

15.1.5. МЕХАНИЗМ РАБОТЫ ПРОТОКОЛА HTTP

Сервер прослушивает TCP-порт в ожидании запросов. Клиент (User-Agent, как правило, браузер) отправляет на сервер запрос. Получив обработав запрос, сервер отправляет клиенту ответ.

Сервер не может по протоколу HTTP отправить ответ, которому не предшествовал запрос. Веб-сервер, получив запрос, в случае, если запрашиваемый документ статический (изображение, или хранимый на диске сервера HTML-документ), считывает документ с диска и отправляет клиенту.

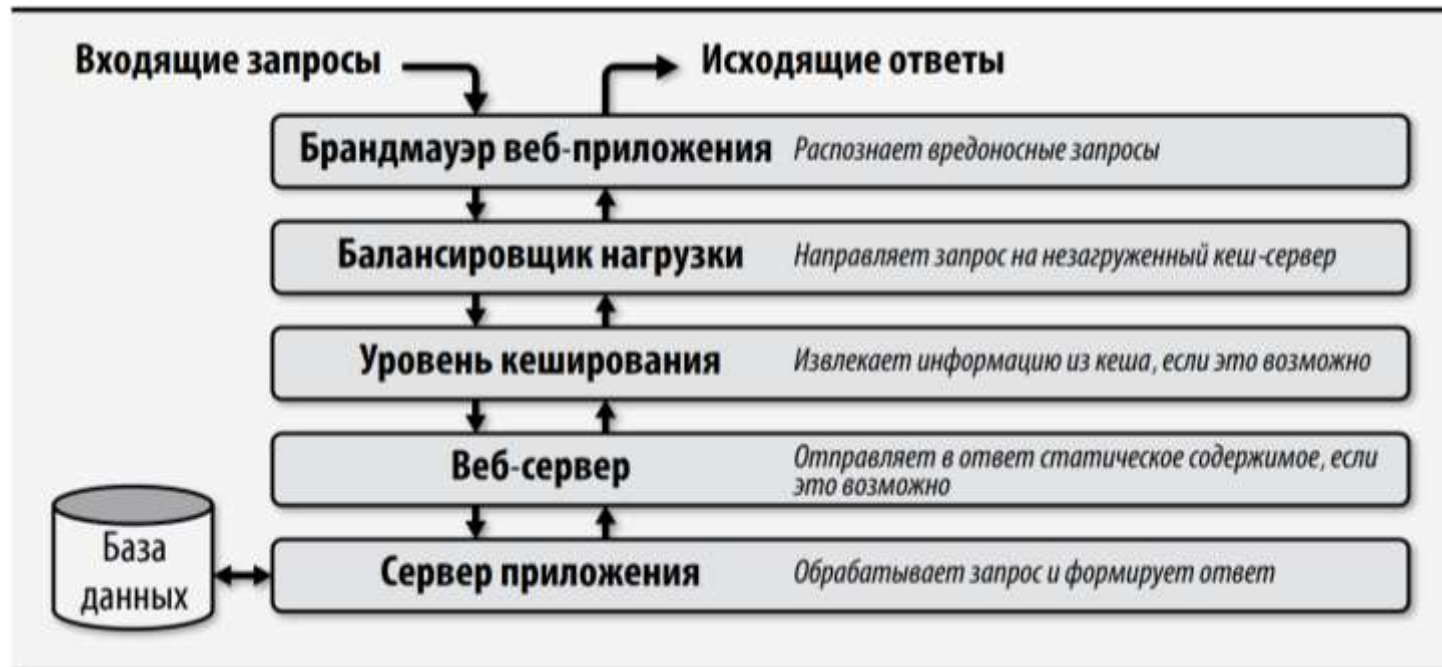
В случае же, если документ – динамический, то веб-сервер запускает сценарий, или передает запрос следующему серверу, умеющему выполнять сценарий. Сценарий генерирует документ и веб-сервер отдает его клиенту (в тех случаях, когда необходимо, чтобы сервер в инициативном порядке отправлял клиенту сообщения, в дополнение к HTTP).



15.1. ПРОТОКОЛ НТТР

15.1.6. РЕАЛИЗАЦИЯ ВЕБ-САЙТОВ И ВЕБ-ПРИЛОЖЕНИЙ

На рисунке показана роль, которую каждая служба и грает в обмене данными по протоколу НТТР. Запросы могут выполняться на верхних уровнях стека, если запрошенный ресурс может быть удовлетворен, или отклоняться с кодом 4xx или 5xx, если возникает проблема. Запросы, требующие запроса к базе данных, проходят все уровни.



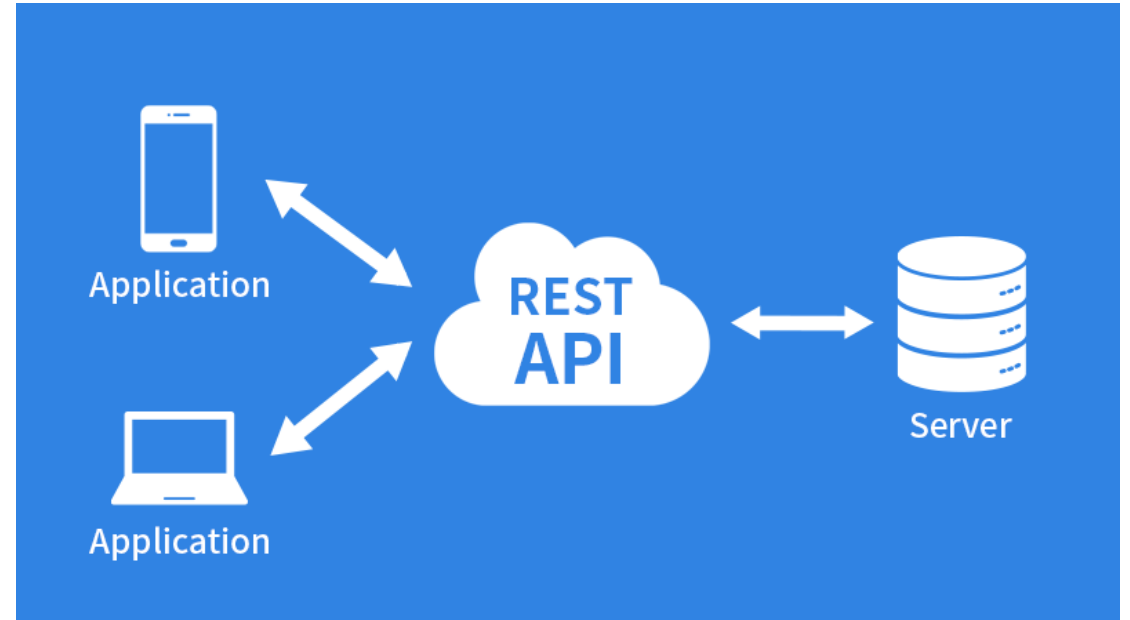
15.1. ПРОТОКОЛ HTTP

15.1.6. РЕАЛИЗАЦИЯ ВЕБ-САЙТОВ И ВЕБ-ПРИЛОЖЕНИЙ

Пользователь в итоге получает документ (например, HTML), который отображается в браузере.

Документ может быть запрошен непосредственно с сервера, или сгенерирован локально SPA (Single Page Application), написанном на javascript.

В случае использования SPA клиент запрашивает у сервера не документ, а данные, используя API (как правило, **REST API**). Получив данные по API, SPA генерирует документ на стороне клиента.



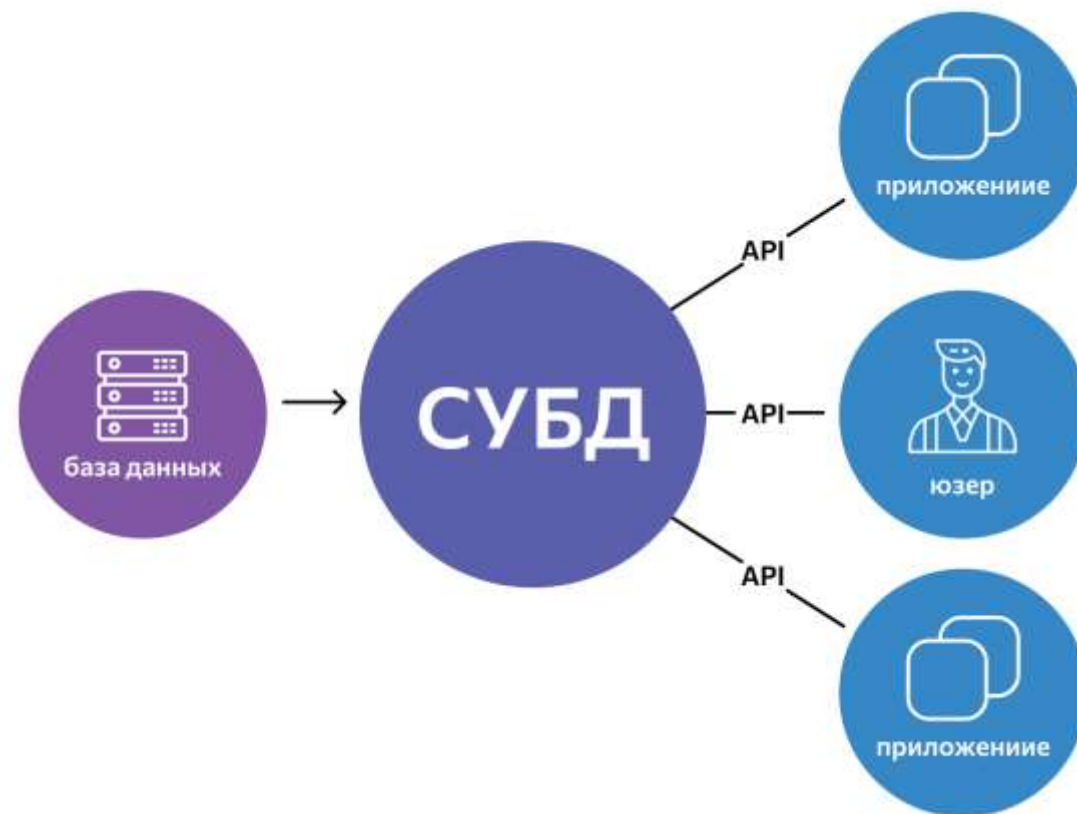
15.1. ПРОТОКОЛ HTTP

15.1.6. РЕАЛИЗАЦИЯ ВЕБ-САЙТОВ И ВЕБ-ПРИЛОЖЕНИЙ

Если запрашивается документ, в зависимости от настройки сервера он может быть считан с диска (статический документ), или сгенерирован на сервере динамически соответствующим сценарием, как правило, с помощью соответствующего модуля.

В случае, если запрос пришел не на получение документа, а на получение данных, он передается соответствующему сценарию, который формирует ответ с содержанием соответствующих данных.

Сценарий для формирования данных может читать данные с диска, обращаться к СУБД, или к другому веб-серверу с применением API.



15.1. ПРОТОКОЛ HTTP

15.1.7. URL И ЕГО СТРУКТУРА

URL-адрес – это идентификатор, определяющий способ и место доступа к ресурсу. URL-адреса не являются HTTP-специфичными; они также используются для других протоколов. Например, мобильные операционные системы используют URL-адреса для облегчения связи между приложениями.

Общий шаблон для URL-адресов имеет вид `схема : адрес`, где `схема` идентифицирует протокол или целевую систему, а `адрес` – это некая строка, которая имеет смысл в рамках этой схемы.

Например, URL `mailto:sa@admin.com` инкапсулирует адрес электронной почты. Если он вызывается в качестве целевой ссылки в Интернете, большинство браузеров будут открывать окно почтового клиента для отправки электронной почты по указанному адресу.

Для Интернета релевантными являются схемы `http` и `https`. На практике можно также встретить схемы `ws` (webSockets), `wss` (webSockets over TLS), `ftp`, `ldap` и многие другие. Адресная часть URL-адреса для веба позволяет определить объемную внутреннюю структуру.

15.1. ПРОТОКОЛ HTTP

15.1.7. URL И ЕГО СТРУКТУРА

Структура URL:

схема : [// [имя [: пароль] @] хост [: порт]] [/ путь] [? параметры] [# якорь]

- схема – как правило, протокол (http, https), но не обязательно (data :, blob : и т.д.);
- имя : пароль – часто применяются при HTTP-авторизации;

Примечание: встраивать пароли в URL-адреса – это плохая идея, потому что URL-адреса могут регистрироваться в журналах, публиковаться в открытых источниках, помещаться в закладки, они видны в результатах работы команды ps и т.д.



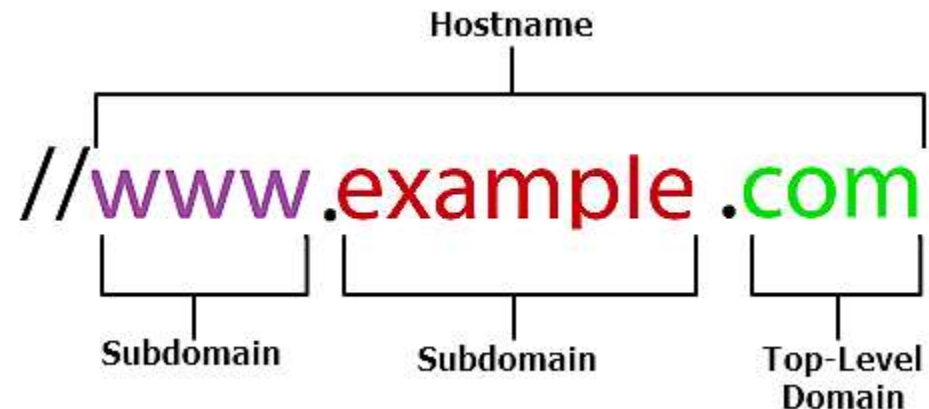
15.1. ПРОТОКОЛ HTTP

15.1.7. URL И ЕГО СТРУКТУРА

Пользовательские агенты могут получать свои учетные данные из источника, отличного от URL-адреса, и обычно это лучший вариант. В веб-браузере достаточно просто не указывать учетные данные и позволить ему запрашивать их отдельно.

- хост — доменное имя или IP-адрес, идентифицирующее веб-сервер;
- порт — указывается, если нестандартный (8000, 8080);

...



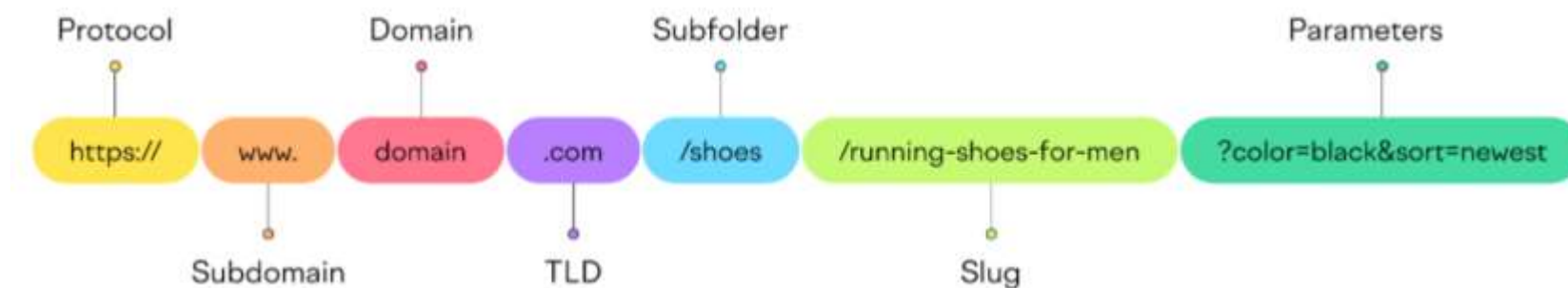
15.1. ПРОТОКОЛ HTTP

15.1.7. URL И ЕГО СТРУКТУРА

- путь – путь к файлу, сценарию, или «виртуальная иерархия» ресурсов, разбираемая сценарием;
- параметры – как правило GET-параметры;
- якорь – позволяет идентифицировать часть документа.

Запрос GET передает данные в параметрах, потому сохраняется в истории.

Запрос POST передает параметры в теле, потому в истории не сохраняются. Однако, в случае применения HTTP, данные в POST легко могут быть перехвачены.

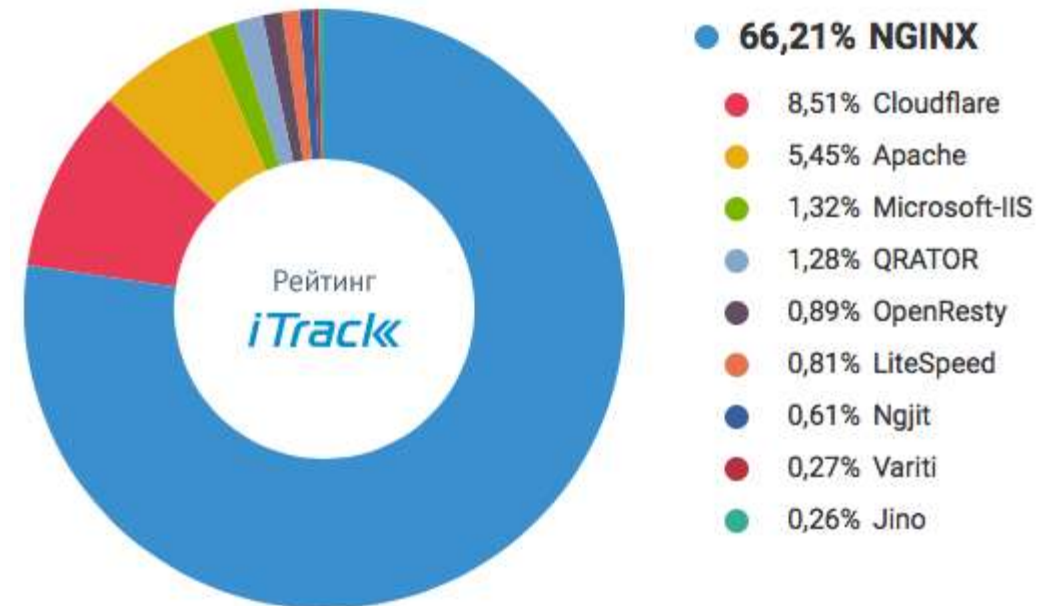


15.1. ПРОТОКОЛ HTTP

15.1.8. ПОПУЛЯРНЫЕ ВЕБ-СЕРВЕРЫ

На конец 2019 года рейтинг самых популярных веб-серверов в рунете возглавлял Nginx. Он шел впереди с огромным отрывом, держа в своих руках более 66% сайтов. После него шел Cloudflare, а тройку лидеров замыкал **Apache**. В 2020 году тенденция не изменилась.

Интересный факт: в мире Nginx тоже оказался самым востребованным веб-сервером, поддерживающим более 479 миллионов сайтов. Но по доле активных сайтов его обгоняет Apache. Поэтому в статистике использования веб-серверов Apache находится на первом месте, а Nginx – на втором.



15.1. ПРОТОКОЛ HTTP

15.1.8. ПОПУЛЯРНЫЕ ВЕБ-СЕРВЕРЫ

Apache был создан как патч для сервера httpd в 1995 году, затем переписан заново (**Apache2**).

Веб-сервер httpd является вездесущим среди множества разновидностей, реализованных в системах UNIX и Linux. Он переносится во многие архитектуры, а готовые пакеты существуют для всех основных систем.

Модульная архитектура является основным фактором успеха Apache. Динамические модули можно подключать с помощью файла конфигурации, предлагая альтернативные варианты аутентификации, повышенную безопасность, поддержку для запуска кода, написанного на большинстве языков, перезапись URL-адресов и многие другие функции.



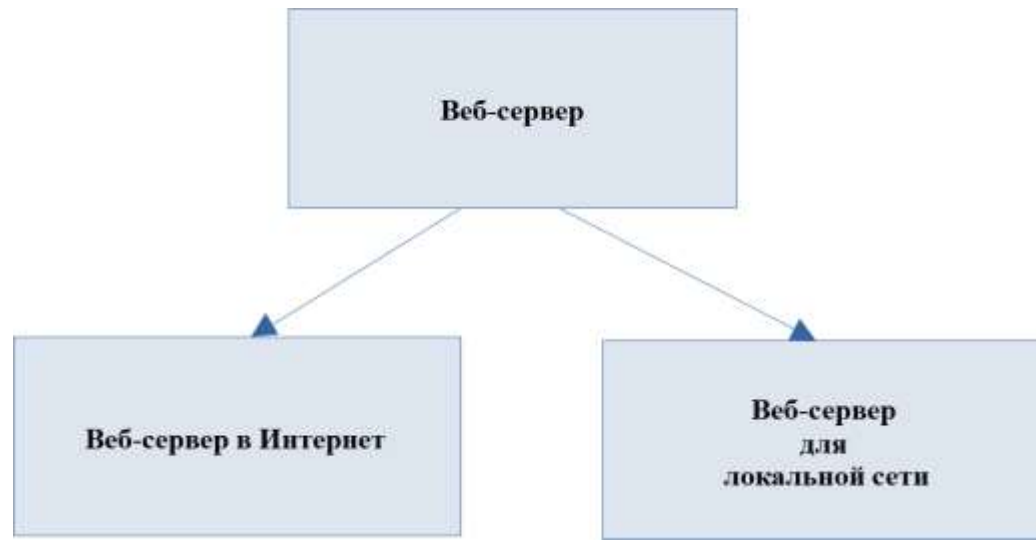
APACHE
HTTP SERVER

15.2. ВЕБ-СЕРВЕР АРАСНЕ

15.2.1. ВЕБ-СЕРВЕР АРАСНЕ В ASTRA LINUX

Веб-сервер `httpd` настраивается с помощью директив в текстовых файлах конфигурации, которые используют отличный синтаксис Apache. Хотя существуют сотни директив, администраторы обычно должны настраивать только несколько.

Директивы и их значения документируются непосредственно в стандартных файлах конфигурации, которые поставляются с ОС, а также доступны на веб-сайте Apache.



15.2. ВЕБ-СЕРВЕР АРАСНЕ

15.2.2. ПРИМЕНЕНИЕ АРАСНЕ В ASTRA LINUX

В Astra Linux в Apache2 есть режим **AstraMode** с поддержкой аутентификации с использованием FreeIPA и мандатного управления доступом.

Если Apache2 применяется для организации веб-сервера для доступа внешних пользователей, то режим AstraMode следует выключить.

Если Apache2 применяется в инфраструктуре предприятия для доступа пользователей локальной сети, с проверкой аутентификации и авторизации с использованием FreeIPA и назначением различных уровней конфиденциальности, то режим AstraMode следует включить.

Параметр AstraMode задается в файле `/etc/apache2/apache2.conf`:

```
DefaultRuntimeDir ${APACHE_RUN_DIR}

# Astra security mode.
#
AstraMode on
```

15.2. ВЕБ-СЕРВЕР АРАСНЕ

15.2.3. УСТАНОВКА И УПРАВЛЕНИЕ АРАСНЕ2

Установка веб-сервера Apache: `apt install apache2`.

Утилита для управления веб-сервером `apachectl`:

`apachectl -v` – версия Apache;

`apachectl -V` – режим работы, опции компиляции;

`apachectl configtest` – проверка корректности настроек;

`apachectl start|stop|restart` – запуск и остановка сервера;

`apachectl graceful|graceful-stop` – «аккуратные» перезапуск и остановка сервера (открытые соединения с сервером не завершаются аварийно);

`systemctl start|stop|reload|status apache2`.

15.2. ВЕБ-СЕРВЕР АРАСНЕ

15.2.3. УСТАНОВКА И УПРАВЛЕНИЕ АРАСНЕ2

Команда `systemctl reload apache2` эквивалентна команде `apachectl graceful`.

В `/etc/apache2/apache2.conf` установить `AstraMode off` (для веб-сервера в сети Интернет).

```
GNU nano 3.2 /etc/apache2/apache2.conf
#ServerRoot "/etc/apache2"
#
# The accept serialization lock file MUST BE STORED ON A LOCAL DISK.
#
#Mutex file:${APACHE_LOCK_DIR} default
#
# The directory where shm and other runtime files will be stored.
#
DefaultRuntimeDir ${APACHE_RUN_DIR}
# Astra security mode.
#
AstraMode off
```

15.2. ВЕБ-СЕРВЕР АРАСНЕ

15.2.4. КОНФИГУРАЦИОННЫЕ ФАЙЛЫ АРАСНЕ2

Конфигурационные файлы файлы apache2:

- `envvars` – переменные окружения для Apache (`APACHE_RUN_USER`, `APACHE_RUN_GROUP` - `www-data`);
- `apache2.conf` – основной конфигурационный файл (глобальные установки);
- `ports.conf` – определяют порты, на которых работает Apache;
- `magic` – сигнатуры для MIME типов для файлов (формат файла).

Конфигурационные файлы находятся в каталоге `/etc/apache2`:

- в каталогах `*available` – доступные настройки;
- в каталогах `*enabled` – включенные настройки (символические ссылки на файлы в каталогах `*available`).

15.2. ВЕБ-СЕРВЕР АРАСНЕ

15.2.4. КОНФИГУРАЦИОННЫЕ ФАЙЛЫ АРАСНЕ2

Каталоги конфигурационных файлов
содержат:

- conf-available, conf-enabled –
дополнительные конфигурационные файлы;

- mods-available, mods-enabled –
установленные модули для Apache
(расширение базовой функциональности);

- sites-available, sites-enabled –
описание виртуальных хостов.

Пример содержимого каталога
/etc/apache2:

```
root@astra:/etc/apache2# ls -la
итого 96
drwxr-xr-x  8 root root  4096 ноя 23 09:33 .
drwxr-xr-x 126 root root 12288 ноя 23 09:33 ..
-rw-r--r--  1 root root  7263 дек  3 2018 apache2.conf
drwxr-xr-x  2 root root  4096 ноя 23 09:33 conf-available
drwxr-xr-x  2 root root  4096 ноя 23 09:33 conf-enabled
-rw-r--r--  1 root root  1782 сен 19 2017 envvars
-rw-r--r--  1 root root 31063 сен 19 2017 magic
drwxr-xr-x  2 root root 12288 ноя 23 09:33 mods-available
drwxr-xr-x  2 root root  4096 ноя 23 09:33 mods-enabled
-rw-r--r--  1 root root   320 сен 19 2017 ports.conf
drwxr-xr-x  2 root root  4096 ноя 23 09:33 sites-available
drwxr-xr-x  2 root root  4096 ноя 23 09:33 sites-enabled
root@astra:/etc/apache2#
```

15.2. ВЕБ-СЕРВЕР АРАСНЕ

15.2.4. КОНФИГУРАЦИОННЫЕ ФАЙЛЫ АРАСНЕ2

Хотя вся конфигурация веб-сервера `httpd` может содержаться в одном файле, разработчики ОС обычно используют директиву `Include` для разделения стандартной конфигурации на несколько файлов и каталогов. Эта архитектура упрощает управление сервером и лучше подходит для автоматизации. Настройки сервера Apache для нескольких платформ, предусмотренные по умолчанию:

	RHEL/CentOS	Debian/Ubuntu	FreeBSD
Имя пакета	<code>httpd</code>	<code>apache2</code>	<code>apache24</code>
Корневой каталог	<code>/etc/httpd</code>	<code>/etc/apache2</code>	<code>/usr/local/etc/apache24</code>
Основной файл конфигурации	<code>conf/httpd.conf</code>	<code>apache2.conf</code>	<code>httpd.conf</code>
Конфигурация модулей	<code>conf.modules.d/</code>	<code>mods-available/</code> <code>mods-enabled/</code>	<code>modules.d/</code>
Конфигурация виртуальных хостов	<code>conf.d/</code>	<code>sites-available/</code> <code>sites-enabled/</code>	<code>Includes/</code>
Расположение журнала	<code>/var/log/httpd</code>	<code>/var/log/apache2</code>	<code>/var/log/httpd-*.log</code>
Пользователь	<code>apache</code>	<code>www-data</code>	<code>www</code>

15.2. ВЕБ-СЕРВЕР АРАСНЕ

15.2.5. ОСНОВНЫЕ ПАРАМЕТРЫ ВИРТУАЛЬНОГО ХОСТИНГА

Большая доля конфигурации веб-сервера `httpd` заключается в определениях виртуальных хостов. Как правило, рекомендуется создавать отдельный файл конфигурации для каждого сайта.

Когда поступает HTTP-запрос, веб-сервер `httpd` определяет, какой виртуальный хост следует выбрать, обратившись к HTTP-заголовку `Host` и сетевому порту. Затем он сопоставляет часть пути запрашиваемого URL-адреса с директивами `Files`, `Directory` и `Location`, чтобы определить, как обслуживать запрошенное содержимое. Этот процесс отображения известен как маршрутизация запросов.



15.2. ВЕБ-СЕРВЕР АРАСНЕ

15.2.5. ОСНОВНЫЕ ПАРАМЕТРЫ ВИРТУАЛЬНОГО ХОСТИНГА

Основные параметры виртуального хоста:

1. `ServerName` – FQDN (full quality domain name), на которое отвечает виртуальный хост.
2. `ServerAlias` – альтернативное имя сервер, на которое отвечает виртуальный хост.
3. `ServerAdmin` – email для сообщения об ошибках.
4. `Listen` – порт (и опционально IP-адрес), на котором работает сервер (должен присутствовать в `ports.conf`).
5. `DocumentRoot` – имя верхнего каталога, в котором находится содержимое веб-сервера.

15.2. ВЕБ-СЕРВЕР АРАСНЕ

15.2.5. ОСНОВНЫЕ ПАРАМЕТРЫ ВИРТУАЛЬНОГО ХОСТИНГА

Настройки виртуального сервера по умолчанию (отвечает на любые запросы, соответствующие данному серверу – IP-адресу и порту, не зависимо от того, какой виртуальный хост указан в заголовке HTTP-запроса Host) – `sites-available/000-default.conf`.

Аналогичный файл для HTTPS – `sites-available/default-ssl.conf`.

Содержимое `sites-available/000-default.conf`:

```
1 VirtualHost *:80
2     # The ServerName directive sets the request scheme, hostname and port that
3     # the server uses to identify itself. This is used when creating
4     # redirection URLs. In the context of virtual hosts, the ServerName
5     # specifies what hostname must appear in the request's Host: header to
6     # match this virtual host. For the default virtual host (this file) this
7     # value is not decisive as it is used as a last resort host regardless.
8     # However, you must set it for any further virtual host explicitly.
9     ServerName www.example.com
10
11     ServerAdmin webmaster@localhost
12     DocumentRoot /var/www/html
```

15.2. ВЕБ-СЕРВЕР АРАСНЕ

15.2.6. НАСТРОЙКА ВИРТУАЛЬНОГО ХОСТИНГА

Виртуальный хостинг может осуществляться по доменному имени (`full quality domain name`), по IP-адресу, по номеру порта.

Настройки для каждого виртуального хоста принято размещать в отдельном файле (с суффиксом `.conf`) в каталоге `sites-available`, для активации виртуального сайта создается символьная ссылка в каталоге `sites-enabled`.

Для активации и деактивации сайта можно использовать команды:

```
a2ensite имя_файла_с_настройками
```

```
a2dissite имя_файла_с_настройками
```

Чтобы отключить файл с настройками по умолчанию, можно воспользоваться командой:

```
a2dissite 000-default.conf
```

Настройки (директивы) виртуального сайта помещаются внутрь контейнера

```
<Virtualhost IP:порт> ...  
</Virtualhost>
```

15.2. ВЕБ-СЕРВЕР АРАСНЕ

15.2.6. НАСТРОЙКА ВИРТУАЛЬНОГО ХОСТИНГА

Пример таких настроек:

```
root@astra:/etc/apache2/sites-available# cat test.ru.conf
<VirtualHost *:80>
    ServerName test.ru
    ServerAlias www.test.ru
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/test.ru
    DirectoryIndex index.html
</VirtualHost>
```

Чтобы пример заработал, необходимо, чтобы локальный DNS-сервер выдавал ресурсные записи A (или CNAME) для `test.ru` и `www.test.ru`, или хотя бы соответствие имени и адреса должны быть включены в `/etc/hosts` на компьютере, откуда будет производиться обращение.

15.2. ВЕБ-СЕРВЕР АРАСНЕ

15.2.6. НАСТРОЙКА ВИРТУАЛЬНОГО ХОСТИНГА

Apache анализирует заголовок `Hosts`: HTTP-запроса и сравнивает его с именами, указанными в

`ServerName` и `ServerAlias`, иначе решение принимается на основании IP-адреса и TCP-порта, на которые пришел запрос на соединение.

Непосредственно после установки `apache2` активирован виртуальный хост `000-default` (без `ServerName`)

```
root@astra:/etc/apache2/sites-available# cat test.ru.conf
<VirtualHost *:80>
    ServerName test.ru
    ServerAlias www.test.ru
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/test.ru
    DirectoryIndex index.html
</VirtualHost>
```

15.2. ВЕБ-СЕРВЕР АРАСНЕ

15.2.7. МОДУЛИ ДЛЯ АРАСНЕ2

Модули могут быть подключены к Apache во время компиляции или динамически при запуске сервера:

- `apachectl -l` – список скомпилированных модулей;
- `apt search libapache2` – список внешних модулей доступных в репозитории.

Установленные модули и конфигурационные файлы для модулей находятся в каталоге `mods-available/`, активированные модули (символьные ссылки) в каталоге `mods-enabled/`.

Активация и деактивация модулей может быть выполнена командами:

```
a2enmod имя_модуля
```

```
systemctl restart apache2
```

```
a2dismod имя_модуля
```

```
systemctl restart apache2
```

ВОПРОСЫ ДЛЯ ПРОВЕРКИ



1. Если Apache2 применяется для организации веб-сервера для доступа внешних пользователей, какой параметр необходимо поменять в файле конфигурации Apache2?
2. На каком порту работает виртуальный хостинг по умолчанию?
3. Что нужно сделать после изменения конфигурационного файла?
4. Что нужно сделать после добавления модуля?
5. Каким образом следует активировать сайт, описанный в конфигурационном файле `example.com.conf`?